



1º Trabalho Prático

Assunto : Ordenação e Análise de Complexidade

Metodologia de Ensino

Nesse trabalho, a metodologia de ensino utilizada é a **Aprendizagem Baseada em Problemas** (ABP). A ABP é uma metodologia de ensino ativa e, nesta abordagem, os alunos são apresentados a um problema desafiador, que requer a aplicação de conceitos e habilidades aprendidas previamente. Os alunos são incentivados a trabalhar em equipe, colaborar, investigar, analisar informações relevantes, formular hipóteses, desenvolver estratégias de resolução e tomar decisões informadas para encontrar soluções para o problema apresentado.

Esse trabalho deverá ser desenvolvido em grupo formado por 4 (cinco) pessoas.

Problema: Auditoria e Benchmarking de Algoritmos de Ordenação

Você e sua equipe foram contratados por uma consultoria de qualidade de software (QA) para auditar uma biblioteca de ordenação desenvolvida por terceiros. A empresa cliente suspeita que algumas implementações da biblioteca `ordenacao.h` apresentam inconsistências, seja em termos de desempenho, contagem de operações, que violam as expectativas teóricas de complexidade e estabilidade.

O papel da equipe não é implementar os algoritmos, mas sim executar experimentos, analisar resultados e identificar possíveis falhas nas implementações fornecidas. Para isso, a equipe deverá desenvolver um sistema de *benchmarking* para extrair métricas reais da biblioteca.

No link abaixo há um exemplo de *benchmarking* desenvolvido na linguagem java. Nesse exemplo os autores utilizaram apenas diferentes tamanhos de conjuntos de dados.

<https://github.com/arisath/Benchmarking-Sorting-Algorithms>

Descrição do problema:

O sistema de benchmarking deve avaliar os seguintes algoritmos (fornecidos na biblioteca):

- Bolha Inteligente
- Inserção
- Seleção
- MergeSort
- QuickSort Tradicional
- ShellSort
- HeapSort



O sistema deve ser capaz de receber conjuntos de dados de diferentes tamanhos e ordenações (aleatório, ordem crescente, quase ordenados (10% fora da ordem) e ordem decrescente) e executar cada algoritmo de ordenação sobre esses conjuntos.

Cenários de Teste

O sistema deve executar cada algoritmo sobre conjuntos de dados (structs de "Candidatos" com Nota e ID) nos seguintes cenários:

- **Aleatório:** Dados sem ordem aparente.
- **Ordenado:** Dados já em ordem crescente.
- **Quase Ordenado:** 90% do vetor ordenado (10% fora de ordem).
- **Inversamente Ordenado:** Dados em ordem decrescente.

Métricas de Desempenho

Para cada execução, o sistema deve capturar e exibir:

- **Número de Comparações de Chaves.**
- **Número de Movimentações de Registros:** (Contabilizando acessos de escrita no vetor principal e auxiliares).
- **Tempo Total de Execução:** (Medido em milissegundos/microssegundos).
- **Verificação de Estabilidade:** O sistema deve checar se, em caso de notas iguais, a ordem original dos IDs foi preservada.

Tarefas

1. **Projetar e implementar um sistema de *benchmarking* que seja capaz de receber conjuntos de dados de entrada e executar os algoritmos de ordenação especificados.**
 - O sistema deve ser implementado utilizando a linguagem C.
 - Deve ser modularizado e incluir as boas práticas de programação (código limpo, bem comentado e indentado).
 - Cada teste deve contemplar um tamanho de vetor e quatro diferentes ordenações: aleatório, ordem crescente, ordem decrescente, quase ordenado.
 - Para os conjuntos aleatório e quase ordenado, o resultado do teste deve ser uma média de 30 repetições com conjuntos diferentes.
 - Ao final dos testes, o sistema deverá gerar uma tabela para cada métrica de avaliação, conforme exemplo da Tabela 1, que considera o tempo de processamento.



Tabela 1: Exemplo de saída do *benchmarking* para a métrica tempo de processamento.

Teste	Tamanho Entrada	Tempo (ms)			
		Aleatório (média de 5 repetições)	Ordem crescente	Ordem decrescente	Quase ordenado (média de 5 repetições)
1	1000				
2	10000				
3	100000				
4	1000000				
5	10000000				

2. Auditoria e Diagnóstico de Anomalias

Além da implementação do sistema de *benchmarking*, cada equipe deverá atuar como auditores de *Quality Assurance* (QA). A biblioteca fornecida (ordenacao.h) contém uma implementação propositalmente ineficiente ou tecnicamente incorreta em um dos algoritmos.

As sub-tarefas de auditoria são:

- **Identificação da Anomalia:** Com base nas tabelas geradas e nos gráficos de crescimento, o grupo deve identificar qual algoritmo não está performando de acordo com sua complexidade teórica ($O(n \log n)$ ou $O(n^2)$ ou qual falhou no teste de estabilidade.
- **Análise de Causa Raiz:** O grupo deve formular uma hipótese sobre o que está causando o "bug".
 - *Exemplo:* "O algoritmo QuickSort apresentou tempo quadrático no cenário X, o que sugere uma escolha de pivô inadequada para este conjunto de dados."
- **Relatório de Não-Conformidade:** O grupo deve descrever formalmente o comportamento esperado (Teoria) vs. o comportamento observado (Prática), justificando o erro através das métricas de comparações e movimentações.

3. Implementação e Validação da Versão Corretiva

Após identificar qual algoritmo da biblioteca ordenacao.h apresenta comportamento anômalo ou ineficiente, o grupo deverá realizar as seguintes tarefas:

- **Desenvolvimento da Versão "Pro" (Correta):** O grupo deve implementar, do zero, a sua própria versão do algoritmo que foi identificado com erro. Esta implementação deve seguir os padrões clássicos da literatura para garantir a melhor complexidade assintótica possível e a estabilidade (se aplicável).
- **Reexecução do Benchmarking:** O sistema de benchmarking deve ser atualizado para incluir a "Versão do Grupo" no comparativo.



- **Análise de Ganho de Performance:** O grupo deve gerar um novo gráfico comparando especificamente a Versão Ineficiente (Biblioteca) vs. Versão Corrigida (Grupo).
- **Prova de Estabilidade:** Caso o erro identificado tenha sido de instabilidade, o grupo deve apresentar os logs do sistema provando que a nova versão preserva a ordem original dos IDs em chaves empatadas.

Investigação Experimental do BozoSort

A biblioteca fornecida inclui a implementação do algoritmo BozoSort, um método de ordenação baseado em trocas aleatórias. Ele funciona escolhendo dois índices aleatórios do vetor e trocando-os. Se o vetor estiver ordenado, ele para. Se não, repete o processo.

A biblioteca que você recebeu contém um algoritmo experimental chamado bozosort. A empresa quer saber se ele é viável para conjuntos de dados extremamente pequenos (ex: microchips de IoT).

4. **Projetar e implementar um sistema de *benchmarking* que seja capaz de receber conjuntos de dados de entrada e executar os algoritmos de ordenação especificados.**
 - Execute o BozoSort para vetores de tamanho 4, 8, 10 e 12 elementos
 - Para cada tamanho, realizar 5 repetições e armazenar as métricas
 - Estabeleça um tempo limite de 30 segundos. Se o algoritmo não terminar, interrompa.
 - Descubra qual o maior tamanho de vetor que seu computador consegue ordenar em menos de 5 minutos usando este método.
5. **Análise de comportamento**
 - Como o tempo cresce com o tamanho da entrada?
 - Por que o tempo de execução do Bozosort varia tanto entre uma execução e outra para o mesmo tamanho de vetor?
 - Discuta viabilidade do BozoSort considerando a curva de complexidade assintótica gerada

Papeis e Responsabilidades

Em atividades que utilizam a Abordagem Baseada em Problemas, é importante que haja a divisão de papeis e responsabilidades. Ao ter papeis e responsabilidades bem definidos, espera-se que haja uma colaboração eficaz e que as tarefas sejam concluídas de forma eficiente e responsável. Isso contribui para o sucesso geral do projeto e para o desenvolvimento das habilidades de trabalho em equipe dos membros da equipe.

No entanto, a abordagem baseada em problemas valoriza a contribuição de todos os participantes, e é importante que todos tenham a oportunidade de se envolver ativamente na investigação, análise e discussão do problema. Nesse trabalho, em especial, é importante que todos compreendam a lógica dos algoritmos estudados e se envolvam no desenvolvimento do código.

Para esse trabalho, são sugeridos os seguintes papéis e responsabilidades:



1. **Líder do Grupo e Arquiteto de Testes:** Este integrante é o responsável direto pela coordenação das atividades e pelo cumprimento do cronograma de testes nos diferentes cenários. Ele define a arquitetura técnica do sistema de benchmarking em C, garantindo a correta modularização e a integração entre o código desenvolvido pelo grupo e a biblioteca externa fornecida pelo professor. Além disso, atua como facilitador da comunicação interna e controla as versões do software para assegurar que todos os componentes e algoritmos estejam operando em harmonia durante as rodadas de execução.
2. **Cientista de Dados e Analista de Desempenho:** O foco deste papel é o tratamento estatístico e a interpretação dos resultados coletados pelo sistema. Ele deve assegurar a validade das médias nas 30 repetições de cada teste, eliminando interferências externas que possam distorcer o tempo de execução. Sua principal entrega é a transformação de dados brutos em gráficos de complexidade assintótica, comparando o crescimento real das métricas com as curvas teóricas da literatura para validar se cada algoritmo está operando dentro da notação Big-O esperada.
3. **Auditor e Desenvolvedor de Correções:** Este membro atua para identificar possíveis falhas ou comportamentos inesperados nas implementações, sendo encarregado de analisar os comportamentos anômalos detectados nos gráficos e formular hipóteses técnicas sobre a causa raiz das falhas na biblioteca. Após o diagnóstico, ele lidera a implementação da versão corrigida do algoritmo defeituoso, aplicando as melhores práticas de estrutura de dados para restaurar a eficiência e a estabilidade. Ao final, deve apresentar evidências técnicas que comprovem a superioridade da nova implementação sobre a versão sabotada.
4. **Registrador e Documentador Técnico:** Responsável por documentar as descobertas do grupo, registrar ideias, anotar *insights*, manter registros claros das discussões e contribuições de cada membro e preparar relatórios ou apresentações. Ademais, fica responsável por escrever documentação técnica detalhada para o software.

Entregas

O grupo deverá entregar, via SIGAA, o relatório conforme modelo do Anexo A. O código desenvolvido deve ser disponibilizado em um repositório de códigos, como o GitHub. O link para o repositório deve ser disponibilizado no relatório. O relatório deve ser entregue, impreterivelmente, até o dia **30/04, meio-dia**.

Além do relatório, o grupo deverá apresentar seu trabalho para a professora. Haverá um sorteio da ordem de apresentação no dia 30/04.



Nota

A nota do trabalho é uma média ponderada das notas obtidas nos itens da rubrica de avaliação apresentados na Tabela 3.

Caso o grupo tenha alguma dúvida sobre o que se espera em algum dos itens, devem questionar antes do prazo de entrega do trabalho.

Não há negociação no prazo de entrega. Trabalhos não entregues até a data serão zerados. Trabalhos entregues fora do SIGAA serão zerados. O resultado da avaliação é entregue pela professora no SIGAA.



Rubricas de Avaliação

Tabela 2: Rubrica de avaliação.

Item Avaliado	Peso	Atendeu Totalmente	Atendeu Parcialmente	Não Atendeu
Precisão do Diagnóstico	3	O grupo identificou corretamente o algoritmo e o erro embutido? A justificativa técnica é fortemente embasada. Nota:10,0	O grupo identificou corretamente o algoritmo e o erro embutido? A justificativa técnica é fraca Nota: 7,0	O grupo não identificou o erro Nota: 0,0 Nesse caso, os demais itens não serão avaliados.
Qualidade da Análise Numérica	2	As tabelas estão completas? Os gráficos são legíveis e mostram as curvas de complexidade de forma clara? Há ligação da teoria com a prática obtida? Nota: 10,0	Tabelas e gráficos de qualidade ruins Nota: 5,0	Tabelas incompletas ou ausência, mesmo que parcial, dos gráficos Nota: 0,0
Eficácia da Correção	2	A versão implementada pelo grupo é realmente corrige o problema? Há evidências numéricas da correção? Nota: 10,0		A versão implementada pelo grupo não corrige o problema ou não há evidências numéricas da correção Nota: 0,0



Análise de Viabilidade do Bozosort	1	A equipe define com precisão até que tamanho de vetor o algoritmo é executável em tempo útil, apresenta o cálculo ou estimativa probabilística de crescimento e justifica a sua viabilidade em sistemas de produção com base em evidências numéricas (tempo/comparações). Nota: 10,0		Não realiza o teste ou não apresenta conclusões sobre a inviabilidade do algoritmo. Nota: 0,0
Comunicação Técnica e Documentação <i>1 ponto extra para o grupo que fizer o relatório em latex, utilizando o modelo de artigo do curso¹</i>	2	A equipa domina os conceitos de complexidade assintótica, responde com segurança às perguntas sobre o "bug" encontrado e utiliza suportes visuais (gráficos) de forma estratégica. O documento segue a estrutura sugerida, utiliza terminologia técnica correta, possui gráficos com eixos devidamente legendados e não apresenta erros ortográficos ou de formatação. A apresentação é organizada e cumpre o tempo de 15 minutos. O grupo é didático e resume bem os resultados, ressaltando as conclusões mais importantes. O grupo relata suas	O relatório contém os dados, mas a análise é superficial. A apresentação é meramente descritiva (lê os slides) e demonstra insegurança ao explicar a causa raiz do problema. Nota: 5,0 a 7,0	Relatório desorganizado, gráficos ilegíveis ou ausência de lógica que conecte os testes ao diagnóstico final. Excesso de IA generativa Nota: 0,0

¹ <https://www.overleaf.com/read/fycfdcfxzwfj>



Universidade Federal de Itajubá
Instituto de Matemática e Computação
Algoritmos e Estruturas de Dados II
CTCO02

Vanessa Souza

		impressões com a metodologia ativa e divisão de papéis. Nota: 10,0		
--	--	--	--	--



ANEXO A

MODELO DO RELATÓRIO

1. INTRODUÇÃO

2. METODOLOGIA

- Especificação do hardware e sistema operacional onde os testes foram rodados.
- Confirmação de que as 30 repetições para os casos aleatórios/quase ordenados foram cumpridas.
- Explicação breve de como a estabilidade foi testada via código.

3. RESULTADOS

Apresentação das 3 Tabelas de Benchmarking (Tempo, Comparações e Movimentações) conforme o modelo fornecido. Logo abaixo de cada tabela, deve haver um Gráfico de Linhas que mostre o crescimento da métrica conforme o tamanho do vetor aumenta.

4. LAUDO DE AUDITORIA

Esta é a seção mais importante. O grupo deve dar o diagnóstico de qual algoritmo não corresponde ao esperado com evidência numérica. O grupo também deve formular uma hipótese da causa raiz do problema para a empresa.

5. CORREÇÃO DO ALGORITMO

Apresentar o trecho de código alterado e um gráfico comparativo entre a Versão Ineficiente (Biblioteca) e a Versão Corrigida (Grupo).

6. ANÁLISE DO CASO BOZOSORT

Apresentação das 3 Tabelas de Benchmarking (Tempo, Comparações e Movimentações) conforme o modelo fornecido. Logo abaixo de cada tabela, deve haver um Gráfico de Linhas que mostre o crescimento da métrica conforme o tamanho do vetor aumenta.

Análise de viabilidade do algoritmo para a empresa.

7. CONSIDERAÇÕES FINAIS